

HIVE: Hidden-Evidence Verification for Hallucination Detection in Diffusion Large Language Models

Guoshenghui Zhao¹ Tan Yu² Weijie Zhao¹

¹Rochester Institute of Technology ²NVIDIA Corporation
gz1626@rit.edu tayu@nvidia.com wjz@cs.rit.edu

Abstract

Diffusion large language models generate text through multi-step denoising, where hallucination signals may emerge throughout the trajectory rather than only in the final output. Existing detectors mainly rely on output uncertainty or coarse trace statistics, which often fail to capture the richer hidden dynamics of D-LLMs. We propose HIVE, a hidden-evidence verification framework that extracts compressed hidden evidence from denoising trajectories, selects informative step-layer evidence, and conditions a verifier language model on the selected evidence through prefix embeddings. HIVE produces both a continuous hallucination score from verifier decision logits and structured verification outputs, including hallucination types, evidence pairs, and short rationales. Across two D-LLMs and three QA benchmarks, HIVE consistently outperforms eight strong baselines and achieves up to 0.9236 AUROC and 0.9537 AUPRC. Ablation studies further confirm the importance of hidden-evidence conditioning, learned evidence selection, two-stream evidence representation, and step-layer embeddings. These results suggest that selected hidden evidence from denoising trajectories provides a stronger and more usable hallucination signal than output-only uncertainty or coarse trace statistics.

1 Introduction

Diffusion large language models (D-LLMs) differ fundamentally from autoregressive language models because they generate text through iterative denoising rather than left-to-right next-token prediction [Li et al., 2022, Gong et al., 2023, Lovelace et al., 2023, Nie et al., 2025, Ye et al., 2025]. This generation process changes not only how responses are produced, but also how model errors emerge and evolve.

As illustrated in Figure 1, hallucination signals in D-LLMs may not be fully characterized by the final output alone: they can develop, persist, or fluctuate across denoising steps, suggesting that intermediate trajectory states may provide valuable signals for hallucination detection [Chang et al., 2026, Hemmat et al., 2026, Qian et al., 2026]. These observations motivate the need for hallucination detectors that go beyond output-only uncertainty and explicitly leverage denoising dynamics.

Most existing hallucination detection methods are developed for autoregressive language models and primarily rely on output uncertainty, response consistency under multiple samples, or static hidden-state representations from a single

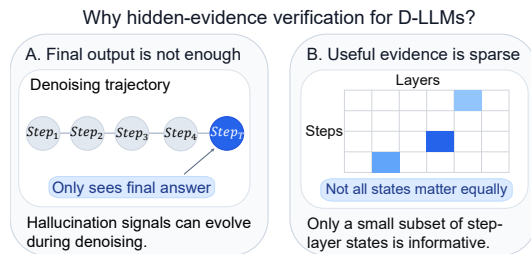


Figure 1: **Motivation for HIVE.** In D-LLMs, hallucination signals may evolve during denoising, and only a sparse subset of step-layer states is informative.

generation pass [Zhang et al., 2025b, Kadavath et al., 2022, Farquhar et al., 2024, Kossen et al., 2024, Manakul et al., 2023, Zhang et al., 2023, Zhao et al., 2024, Su et al., 2024, Chen et al., 2024, Zhang et al., 2025a]. While these signals can be useful, they are not well suited to the structured multi-step denoising process of D-LLMs. In particular, they often overlook that hallucination-relevant evidence may be both temporally evolving and sparsely distributed across intermediate denoising steps and layers. Even when trajectory information is used, recent D-LLM detectors typically summarize trajectory signals at the level of traces, attention graphs, or uncertainty dynamics rather than explicitly identifying the most informative hidden evidence. This leaves an important question underexplored: how can we identify and exploit the most informative hidden evidence from D-LLM denoising trajectories for hallucination detection?

Motivated by these observations, we propose HIVE, a hidden-evidence verification framework for hallucination detection in D-LLMs. HIVE extracts compressed hidden evidence from denoising trajectories, selects informative step-layer evidence, and conditions a verifier language model on the selected evidence through prefix embeddings. This design supports both scalar hallucination detection and structured verification.

Our main contributions are as follows:

- We introduce **HIVE**, a hidden-evidence verification framework for hallucination detection in D-LLMs that operates on denoising trajectories rather than only final outputs.
- We propose a trajectory-aware hidden-evidence pipeline that extracts compressed hidden evidence from denoising trajectories, selects informative step-layer evidence, and injects it into a verifier language model through prefix conditioning.
- We show that hallucination detection in D-LLMs can be framed not only as scalar scoring, but also as structured verification that predicts hallucination types, evidence pairs, and rationales, while preserving a continuous detection signal through verifier decision logits.
- Through experiments on two D-LLMs, three QA benchmarks, eight strong baselines, and comprehensive ablation studies, we show that HIVE consistently achieves the strongest overall hallucination detection performance.

2 Related Work

2.1 Hallucination Detection in LLMs

Existing hallucination detection methods for large language models mainly rely on three types of signals: output uncertainty, response consistency across multiple generations, and internal representations such as hidden states or learned probes [Shapiro et al., 2026, Gupta et al., 2025, Liu et al., 2025, Sawczyn et al., 2026, Jiang et al., 2024, Sriramanan et al., 2024, Chuang et al., 2024, Zhang et al., 2025c, Han et al., 2025]. While effective for autoregressive models, these methods are largely designed around final outputs or single-pass representations and are therefore less naturally aligned with diffusion language models, whose outputs evolve through multi-step denoising.

2.2 Diffusion Language Models and Their Trajectories

Diffusion language models generate text through iterative denoising rather than left-to-right decoding. This produces a structured trajectory in which token identities and internal representations can evolve substantially before the final output is formed [Rossi et al., 2026, Kim et al., 2026, Guo and Tan, 2026, von Rütte et al., 2025]. These trajectory states are therefore not merely intermediate artifacts of generation, but potentially useful signals for reliability analysis.

2.3 Hallucination Detection for D-LLMs

Hallucination detection for D-LLMs remains relatively underexplored [Chang et al., 2026, Hemmat et al., 2026, Qian et al., 2026]. Existing trajectory-based approaches show that denoising trajectories contain useful signals for distinguishing hallucinated from non-hallucinated generations, but they typically rely on coarse summaries such as traces, attention graphs, or uncertainty dynamics [Chang et al., 2026, Hemmat et al., 2026, Qian et al., 2026]. HIVE differs from this line of work by

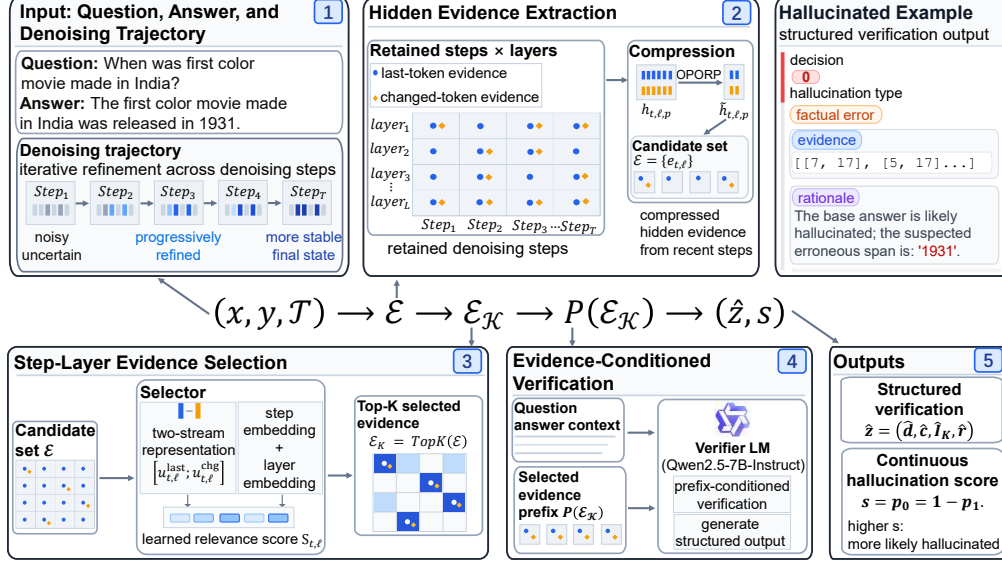


Figure 2: **Overview of HIVE.** Given a question-answer pair and its denoising trajectory, HIVE extracts compressed hidden evidence, selects informative step-layer evidence under a fixed budget, and conditions a verifier language model on the selected evidence through prefix embeddings. The verifier produces both a structured verification output $\hat{z}$ and a continuous hallucination score s . This overall mapping is summarized in Eq. 1. The hallucinated example on the top right illustrates the generated decision, hallucination type, evidence pairs, and rationale.

explicitly selecting informative hidden evidence from denoising steps and layers and using it for evidence-conditioned verification.

3 Method

3.1 Overview of HIVE

Figure 2 summarizes HIVE. Given a question-answer pair (x, y) generated by a D-LLM and its denoising trajectory $\mathcal{T}$, HIVE detects hallucinations by operating on trajectory-level hidden evidence rather than relying only on the final answer. The core idea is that hallucination-relevant signals are often sparse and distributed across denoising steps and layers, so effective detection requires both extracting hidden evidence from $\mathcal{T}$ and identifying which parts of that evidence are most informative.

HIVE consists of four components. First, it extracts compressed hidden evidence from intermediate denoising states, producing a set of candidate evidence units $\mathcal{E}$. Second, it applies a step-layer selector to score these units and retains a top- K subset $\mathcal{E}_K \subset \mathcal{E}$ under a fixed evidence budget. Third, it conditions a verifier language model on the selected evidence through prefix embeddings, producing a structured verification output $\hat{z}$. Finally, it converts the verifier’s decision-emission logits into a scalar hallucination score s .

Overall, HIVE maps an input pair (x, y) and its denoising trajectory $\mathcal{T}$ to both a structured verification output and a continuous detection score:

$$(x, y, \mathcal{T}) \longrightarrow \mathcal{E} \longrightarrow \mathcal{E}_K \longrightarrow (\hat{z}, s). \quad (1)$$

The following subsections describe hidden evidence extraction, step-layer evidence selection, evidence-conditioned verification, and decision-logit hallucination scoring.

3.2 Hidden Evidence Extraction

Let $\mathcal{T}$ denote the denoising trajectory associated with (x, y) , and let $h_{t,\ell,p} \in \mathbb{R}^d$ denote the hidden state at denoising step t , layer ℓ , and token position p .

For each retained denoising step t , we define a candidate position set $\mathcal{P}_t = \{p_t^{\text{last}}\} \cup \mathcal{P}_t^{\text{chg}}$, where p_t^{last} is the last token position of the current answer sequence and $\mathcal{P}_t^{\text{chg}}$ contains positions whose decoded token identities differ from those at the previous denoising step. We retain only the last few denoising steps and cap the number of changed positions per step.

For each retained step t and layer ℓ , hidden states at positions in $\mathcal{P}_t$ are compressed into a lower-dimensional evidence space: $\tilde{h}_{t,\ell,p} = \Pi_\ell(h_{t,\ell,p}) \in \mathbb{R}^r$, where $\Pi_\ell(\cdot)$ maps the original hidden state from dimension d to a compressed evidence dimension $r \ll d$. In practice, we use OPORP-style random projection.

We then define a candidate evidence unit for each step-layer pair (t, ℓ) as $e_{t,\ell} = \{\tilde{h}_{t,\ell,p} \mid p \in \mathcal{P}_t\}$, and collect all retained units into $\mathcal{E} = \{e_{t,\ell} \mid t \in \mathcal{S}, \ell \in \mathcal{L}\}$, where $\mathcal{S}$ is the set of retained denoising steps and $\mathcal{L}$ is the set of considered layers. Each $e_{t,\ell}$ serves as a candidate evidence unit for downstream selection.

3.3 Step-Layer Evidence Selection

The candidate set $\mathcal{E}$ is still too large to be passed directly to the verifier, so HIVE learns a step-layer selector that scores each candidate unit $e_{t,\ell} \in \mathcal{E}$ and retains the most informative ones under a fixed evidence budget.

To make each candidate unit comparable across steps and layers, we summarize $e_{t,\ell}$ with a two-stream representation. Recall that each unit contains compressed hidden states from the last-token position and the changed-token positions at denoising step t . We define $u_{t,\ell}^{\text{last}} = \tilde{h}_{t,\ell,p_t^{\text{last}}} \in \mathbb{R}^r$, and let $u_{t,\ell}^{\text{chg}}$ denote the mean of $\tilde{h}_{t,\ell,p}$ over $p \in \mathcal{P}_t^{\text{chg}}$ when $|\mathcal{P}_t^{\text{chg}}| > 0$, and $\mathbf{0}$ otherwise. We then form the two-stream evidence representation $g_{t,\ell} = [u_{t,\ell}^{\text{last}}; u_{t,\ell}^{\text{chg}}] \in \mathbb{R}^{2r}$.

Because the same hidden pattern can play different roles depending on where it appears in the denoising trajectory, the selector is made step-layer aware. Let $\phi : \mathbb{R}^{2r} \rightarrow \mathbb{R}^{d_{\text{sel}}}$ be a learned projection network, and let $a_t, b_\ell \in \mathbb{R}^{d_{\text{sel}}}$ denote learnable step and layer embeddings, respectively. We project the two-stream feature into a shared selector space and incorporate its step-layer identity: $q_{t,\ell} = \phi(g_{t,\ell}) + a_t + b_\ell$. A scalar relevance score is then computed as $s_{t,\ell} = f_{\text{sel}}(q_{t,\ell})$, where $f_{\text{sel}}(\cdot)$ is a lightweight scoring function.

Let $\mathcal{J} = \{(t, \ell) \mid t \in \mathcal{S}, \ell \in \mathcal{L}\}$ denote the index set of candidate step-layer units in $\mathcal{E}$. To train the selector, we normalize the scores into soft weights

$$w_{t,\ell} = \frac{\exp(s_{t,\ell})}{\sum_{(t', \ell') \in \mathcal{J}} \exp(s_{t', \ell'})}, \quad (t, \ell) \in \mathcal{J},$$

and form an aggregated evidence representation $m = \sum_{(t,\ell) \in \mathcal{J}} w_{t,\ell} q_{t,\ell}$. A lightweight hallucination classifier is trained on m using supervision from the hallucination labels, so that the selector learns to assign higher scores to evidence units that are more predictive of hallucinated versus non-hallucinated generations. In practice, we use a differentiable top- K training scheme with an auxiliary regularization term to encourage stable and informative evidence selection; the formulation above provides the corresponding soft aggregation view.

After training, HIVE uses the learned relevance scores to select the top- K step-layer units under a fixed evidence budget: $\mathcal{I}_K = \text{TopK}(\{s_{t,\ell}\}_{(t,\ell) \in \mathcal{J}})$ and $\mathcal{E}_K = \{e_{t,\ell} \mid (t, \ell) \in \mathcal{I}_K\}$. Thus, $\mathcal{E}_K$ contains the K highest-scoring evidence units according to the learned selector and is passed to the downstream verifier.

3.4 Evidence-Conditioned Verification

HIVE conditions a verifier language model on both the original question-answer context and the selected hidden evidence. For each selected evidence unit $e_{t,\ell} \in \mathcal{E}_K$, we compute an evidence-conditioned prefix representation $v_{t,\ell} = \psi(e_{t,\ell}) \in \mathbb{R}^{d_{\text{ver}}}$, where $\psi(\cdot)$ is an evidence encoder and d_{ver} is the verifier hidden dimension. Collecting all selected units yields a prefix sequence

$$P(\mathcal{E}_K) = [v_{t_1, \ell_1}, \dots, v_{t_K, \ell_K}], \quad (t_j, \ell_j) \in \mathcal{I}_K.$$

This prefix sequence is injected into the verifier through prefix embeddings, while the textual input remains the question-answer pair (x, y) . Conditioned on (x, y) and $P(\mathcal{E}_K)$, the verifier generates a structured verification output $\hat{z} = \text{Ver}(x, y; P(\mathcal{E}_K))$, where $\hat{z}$ contains a binary hallucination decision, a predicted hallucination type, selected evidence pairs, and a short rationale. Concretely, we write $\hat{z} = (\hat{d}, \hat{e}, \hat{\mathcal{I}}_K, \hat{r})$, where $\hat{d} \in \{0, 1\}$ is the verifier decision, $\hat{e}$ is the predicted hallucination type, $\hat{\mathcal{I}}_K$ denotes the referenced evidence pairs, and $\hat{r}$ is a short rationale. During training, the verifier is supervised to generate this structured output format.

3.5 Decision-Logit Hallucination Scoring

In addition to the structured verification output $\hat{z}$, HIVE derives a continuous hallucination score from the verifier’s decision-emission logits. Let z_0 and z_1 denote the logits assigned to the two decision labels at the decision-emission step, where label 0 denotes a hallucinated answer and label 1 denotes a non-hallucinated answer. We convert them into probabilities as $p_1 = \frac{\exp(z_1)}{\exp(z_0) + \exp(z_1)}$ and $p_0 = \frac{\exp(z_0)}{\exp(z_0) + \exp(z_1)}$. We then define the scalar hallucination score as $s = p_0 = 1 - p_1$. Higher values of s therefore indicate higher confidence that the answer is hallucinated. This continuous score enables threshold-free metrics such as AUROC and AUPRC and preserves discriminative information that is lost after hard thresholding.

4 Experimental Setup

4.1 Models and Benchmarks

We evaluate HIVE on two representative D-LLMs, Dream-7B-Instruct and LLaDA-8B-Instruct, to test whether HIVE generalizes across different diffusion language model families.

We evaluate on three QA benchmarks with complementary characteristics: TriviaQA, HotpotQA, and NQOpenLike [Joshi et al., 2017, Yang et al., 2018, Kwiatkowski et al., 2019]. TriviaQA provides a relatively clean open-domain QA setting, HotpotQA emphasizes multi-hop reasoning, and NQOpenLike introduces a more skewed label distribution that makes threshold-free metrics especially informative [Davis and Goadrich, 2006].

For each model-benchmark pair, we run the target D-LLM to obtain generated answers and their associated denoising trajectories, and then evaluate hallucination detection on the resulting question-answer examples. This yields a total of six evaluation settings spanning two D-LLMs and three QA benchmarks.

4.2 Baselines

We compare HIVE against eight strong baselines spanning output-based, latent-based, and trajectory-based hallucination detection.

Output-based baselines. We include Perplexity, LN-Entropy, Semantic Entropy, and Lexical Similarity. These methods estimate hallucination risk from the generated outputs themselves, without explicitly using intermediate hidden trajectories. Perplexity and LN-Entropy rely on token-level or sequence-level uncertainty signals [Malinin and Gales, 2021, Lin et al., 2024]. Semantic Entropy and Lexical Similarity, by contrast, measure the consistency of multiple sampled answers at the semantic or lexical level [Kuhn et al., 2023].

Latent-based baselines. We include EigenScore, CCS, and TSV, which operate on internal model representations rather than output-only uncertainty [Burns et al., 2023, Park et al., 2025]. These methods test whether hidden-state geometry or learned latent probes can provide stronger hallucination signals than output-level statistics alone [Azaria and Mitchell, 2023, Du et al., 2024].

Trajectory-based baseline. We include TraceDet, a recent diffusion-language-model hallucination detector that operates on decoding trajectories. TraceDet is the most directly related baseline in our comparison, as it also exploits trajectory-level signals in D-LLMs. Compared with TraceDet, HIVE focuses on selecting informative hidden evidence from denoising steps and layers and using that evidence for verifier-based hallucination assessment.

Method	Dream-7B-Instruct			LLaDA-8B-Instruct		
	HotpotQA	NQOpenLike	TriviaQA	HotpotQA	NQOpenLike	TriviaQA
Semantic Entropy	81.84 / 93.10	72.17 / 91.47	81.92 / 86.92	78.00 / 90.04	81.24 / 94.57	80.98 / 86.79
CCS	89.60 / 96.65	81.31 / 92.98	88.75 / 91.22	87.79 / 92.98	81.46 / 95.09	84.33 / 89.95
TraceDet	87.40 / 95.64	79.47 / 94.86	83.44 / 88.60	86.05 / 94.28	77.68 / 94.16	80.07 / 86.89
HIVE	91.76 / 97.37	85.74 / 96.60	92.36 / 95.37	90.94 / 96.48	85.71 / 96.95	90.66 / 93.98

Table 1: **Main overall results.** Comparison against the strongest output-based, latent-based, and trajectory-based baselines across two D-LLMs and three QA benchmarks. Each cell reports AUROC / AUPRC (%), higher is better). Full results for all baselines are reported in Appendix Table 6.

4.3 Evaluation Metrics

We evaluate HIVE from three complementary perspectives: threshold-free detection quality, thresholded classification performance, and structured verification quality. For methods that produce continuous hallucination scores, we report AUROC and AUPRC, which measure how well hallucinated answers are ranked above non-hallucinated ones without fixing a decision threshold; for HIVE, this score is derived from the verifier’s decision-emission logits (Section 3.5) [Davis and Goadrich, 2006]. We additionally report thresholded classification metrics including F1, Accuracy, Balanced Accuracy, and Specificity, where the decision threshold is selected on the validation split and transferred to the test split; Balanced Accuracy and Specificity are particularly useful for assessing whether a method can reliably identify non-hallucinated answers rather than simply follow the majority class [Brodersen et al., 2010]. Because HIVE also produces structured verification outputs, we further report valid JSON rate, decision accuracy, hallucination type accuracy, and exact-match metrics for structured fields when applicable. Unless otherwise specified, hallucinated answers are treated as the positive class. For datasets with skewed hallucination label distributions, such as NQOpenLike, we place greater emphasis on AUROC and AUPRC, whereas for more balanced settings thresholded metrics provide an additional view of decision quality.

4.4 Implementation Details

For all experiments, HIVE is implemented as a four-stage pipeline consisting of hidden evidence extraction, step-layer evidence selection, evidence-conditioned verification, and decision-logit scoring. We retain a fixed number of recent denoising steps from each trajectory, construct candidate evidence units from the last-token position together with changed-token positions, and compress hidden states with OPORP-style random projection to dimension $r = 64$ [Li and Li, 2023]. For step-layer evidence selection, we use the two-stream representation in Section 3.3 and select a fixed evidence budget of $K = 16$ step-layer units per example, using all retained layers of the underlying D-LLM trajectory (28 for Dream-7B-Instruct and 32 for LLaDA-8B-Instruct). For evidence-conditioned verification, we use Qwen2.5-7B-Instruct as the verifier backbone [Qwen Team, 2025]; selected evidence units are encoded into prefix embeddings and injected together with the original question-answer context, following continuous prefix conditioning [Li and Liang, 2021]. The verifier is trained to generate structured outputs containing a binary hallucination decision, hallucination type, evidence pairs, and a short rationale, with maximum input length 1024 tokens unless otherwise specified. For evaluation, HIVE derives a continuous hallucination score from the verifier’s decision-emission logits, with thresholds selected on the validation split and transferred to the test split. Additional implementation details, prompt templates, and runtime hyperparameters are provided in Appendix A, Appendix B, and Tables 4–5.

5 Main Results

Table 1 summarizes the main overall comparison across two D-LLMs and three QA benchmarks, while the full nine-method results are reported in Appendix Table 6.

HIVE is the strongest method across all six settings, achieving the best AUROC and AUPRC in every case. The gains are especially clear on TriviaQA and HotpotQA. On Dream-7B-Instruct, HIVE reaches 0.9236 / 0.9537 on TriviaQA and 0.9176 / 0.9737 on HotpotQA (AUROC / AUPRC). On LLaDA-8B-Instruct, the corresponding results are 0.9066 / 0.9398 and 0.9094 / 0.9648. Table 2

Method	Dream-7B-Instruct HotpotQA			Dream-7B-Instruct TriviaQA			LLaDA-8B-Instruct HotpotQA			LLaDA-8B-Instruct TriviaQA		
	F1	BAcc	Spec	F1	BAcc	Spec	F1	BAcc	Spec	F1	BAcc	Spec
CCS	90.53	73.72	53.54	83.94	79.18	74.36	90.30	74.13	53.45	83.58	73.51	58.78
TraceDet	89.78	76.48	62.68	81.92	71.41	52.88	86.23	51.04	2.10	78.25	50.01	0.02
HIVE	91.47	77.77	61.96	87.76	83.63	78.53	91.20	79.39	65.53	88.07	83.27	78.41

Table 2: **Main thresholded results.** Comparison against the strongest latent-based and trajectory-based baselines on HotpotQA and TriviaQA. Each cell reports F1 / Balanced Accuracy / Specificity (%; higher is better). Full thresholded results for all baselines are reported in Appendix Table 7.

further shows that HIVE also delivers the strongest thresholded classification performance against the selected latent-based and trajectory-based baselines on these two benchmarks, indicating that its advantage is not limited to score ordering alone.

On NQOpenLike, HIVE again attains the best AUROC and AUPRC for both Dream and LLaDA, achieving 0.8574 / 0.9660 and 0.8571 / 0.9695, respectively. These settings exhibit substantially more skewed hallucination label distributions, making threshold-free ranking quality particularly informative in comparison with fixed-threshold summaries. We therefore place greater emphasis on AUROC and AUPRC for NQOpenLike, where HIVE consistently provides the strongest hallucination signal.

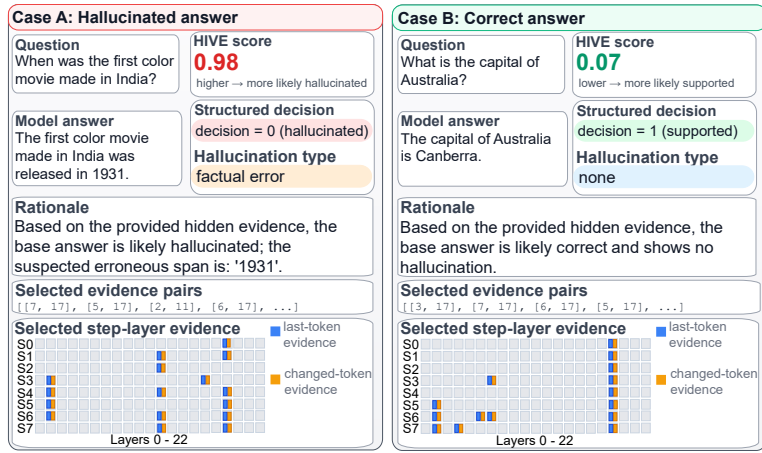


Figure 3: **Qualitative examples of HIVE on a hallucinated answer and a non-hallucinated answer.** For each case, we show the HIVE score, structured decision, hallucination type, rationale, selected evidence pairs, and selected step-layer evidence.

HIVE also produces stable structured verification outputs. As reported in Appendix Table 8, valid-JSON rate remains near perfect across all six settings (99.43–100.00), while decision accuracy ranges from 83.18 to 86.49 and hallucination type accuracy from 61.52 to 72.43. Figure 3 further illustrates representative structured outputs. Overall, HIVE improves both threshold-free hallucination detection and structured verification quality across different D-LLM backbones and benchmark settings.

6 Ablation Studies

We conduct ablations on the showcase setting, Dream-7B-Instruct + TriviaQA, to isolate the contribution of each major design choice in HIVE. Table 3 reports a compact ablation summary, while full per-variant metric breakdowns are deferred to Appendix Table 9. Figure 5 provides a complementary visual summary in terms of AUROC and F1. Overall, the ablations show that HIVE benefits from all five components: hidden-evidence prefix conditioning, learned step-layer evidence selection, two-stream evidence representation, explicit step-layer embeddings, and decision-logit scoring.

6.1 Hidden-Evidence Prefix Conditioning

We first examine whether HIVE’s gains come from hidden-evidence prefix conditioning rather than simply from using a strong verifier language model. To this end, we compare full HIVE against three variants: (1) a plain verifier retrained without evidence prefixing, (2) the prefix-trained verifier

Setting	Best ablated variant	AUROC / F1	Δ AUROC / Δ F1
Full HIVE	–	92.36 / 87.76	–
Prefix conditioning	Shuffled evidence prefix	88.57 / 84.28	-3.79 / -3.48
Evidence selection	Top-final-step-only	89.17 / 85.82	-3.19 / -1.94
Two-stream representation	Lasttok only	90.48 / 86.63	-1.88 / -1.13
Step-layer embeddings	w/o layer embedding	91.43 / 86.62	-0.93 / -1.14
Decision-logit scoring	Binary decision only	– / 86.53	– / -1.23

Table 3: **Main ablation summary.** Compact ablation results on Dream-7B-Instruct + TriviaQA. For each ablation group, we report the strongest ablated variant relative to full HIVE. Full per-variant metric breakdowns are reported in Appendix Table 9.

evaluated without prefix at inference time, and (3) a shuffled-prefix variant that preserves prefix size and format but breaks evidence alignment.

The results show a clear dependence on aligned hidden-evidence prefixes. Relative to full HIVE, AUROC drops from 92.36 to 88.54 for the retained no-prefix verifier, 86.64 for no prefix at inference, and 88.57 for shuffled prefixes; F1 similarly drops from 87.76 to 84.25, 82.26, and 84.28. These results show that HIVE’s gains come from conditioning verification on correctly selected hidden evidence rather than merely using a strong verifier or adding extra prefix tokens.

6.2 Learned Step-Layer Evidence Selection

We next examine whether HIVE’s learned step-layer selector is necessary, or whether similar gains can be obtained with simpler evidence selection strategies under the same evidence budget [Xie et al., 2020]. Figure 4 shows that selected evidence is concentrated on a sparse subset of denoising steps and layers rather than being uniformly distributed, already suggesting that not all step-layer units are equally informative for verification. To test this more directly, we compare full HIVE against three alternatives: (1) *Top-final-step-only*, which restricts the candidate set to the final retained denoising step and then selects the top- K step-layer units within that step using the learned selector scores; (2) *uniform-over-steps*, a non-learned heuristic that distributes the evidence budget approximately evenly across retained denoising steps and then selects layers within each step using a fixed deterministic rule; and (3) *random K -pair selection*, which samples K step-layer units uniformly at random.

The learned selector outperforms all three alternatives. Relative to full HIVE, AUROC drops to 89.17 for top-final-step-only, 88.34 for uniform-over-steps, and 85.85 for random K selection. The strongest non-full variant is top-final-step-only, indicating that the final denoising step contains useful signal, but its gap from the full model shows that informative evidence is not concentrated only in the last step. Together with Figure 4, these results show that HIVE benefits from selecting the right hidden evidence under a fixed budget.

6.3 Two-Stream Evidence Representation

We then study whether HIVE’s two-stream evidence representation is necessary, or whether either stream alone is sufficient. To this end, we compare the full model, which uses the concatenated representation $[u^{\text{last}}; u^{\text{chg}}]$, against two variants: *Lasttok only*, which uses only the last-token representation, and *Changed only*, which uses only the mean-pooled changed-token representation.

Both single-stream variants remain competitive, but the two-stream design is consistently strongest. AUROC drops from 92.36 to 90.48 for Lasttok only and 89.93 for Changed only, with corresponding F1 drops from 87.76 to 86.63 and 85.50. This suggests that global final-state information and local revision dynamics are complementary rather than redundant.

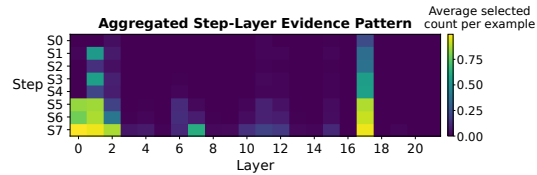


Figure 4: **Aggregated step-layer evidence pattern.** Average selected count per example over all test instances. The selector concentrates evidence on a sparse subset of denoising steps and layers rather than distributing it uniformly.

6.4 Step-Layer Embeddings

We next examine whether explicit step-layer embeddings are necessary for effective evidence selection. To this end, we compare the full selector, which uses both step and layer embeddings, against two variants: one without step embeddings and one without layer embeddings [Tenney et al., 2019, Jawahar et al., 2019].

Removing either embedding degrades performance: AUROC falls to 90.93 without step embeddings and 91.43 without layer embeddings, with similar declines in F1. This indicates that evidence selection depends not only on hidden content but also on where evidence appears in the denoising trajectory, with step identity contributing slightly more in this setting.

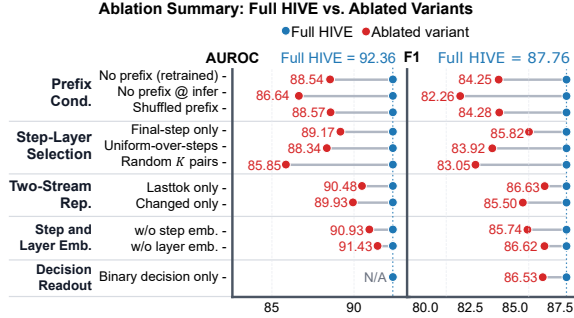


Figure 5: **Ablation summary.** Comparison between Full HIVE and ablated variants on Dream-7B-Instruct + TriviaQA under AUROC and F1.

6.5 Decision-Logit Hallucination Scoring

Decision-logit scoring is more informative than using the final hard decision alone. The binary-decision variant reaches 86.53 F1, 82.69 Balanced Accuracy, 77.95 Specificity, and 84.94 Decision Accuracy, all slightly below the default decision-logit readout. More importantly, only the decision-logit view provides a continuous hallucination score and therefore supports threshold-free metrics such as AUROC and AUPRC.

7 Limitations and Broader Impacts

Limitations. HIVE is evaluated on only two diffusion language model families and three QA benchmarks, so the reported gains may not directly transfer to other D-LLMs or task settings. It also introduces additional computational overhead through trajectory feature extraction and verifier-based scoring, and the current study reports single-run results without quantifying variability across repeated runs.

Broader Impacts. HIVE is intended to improve the reliability and interpretability of diffusion language models through stronger hallucination detection and structured verification, which may be useful in applications where factual consistency matters. However, verifier outputs are not guarantees of correctness and could be over-relied upon in downstream or high-stakes settings, so they should be used with appropriate human oversight.

8 Conclusion

We presented **HIVE**, a hidden-evidence verification framework for hallucination detection in diffusion large language models. By extracting compressed hidden evidence from denoising trajectories, selecting informative step-layer evidence, and conditioning a verifier language model on the selected evidence, HIVE supports both continuous hallucination scoring and structured verification. Across two D-LLMs and three QA benchmarks, HIVE consistently achieved the strongest overall detection performance and produced stable structured outputs. These results suggest that selected hidden evidence from denoising trajectories provides a stronger and more usable hallucination signal than output-only uncertainty or coarse trace statistics, highlighting the value of trajectory-level analysis for D-LLM reliability.

References

Amos Azaria and Tom Mitchell. The internal state of an LLM knows when it’s lying. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 967–976, Singapore, 2023.

- Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.68. URL <https://aclanthology.org/2023.findings-emnlp.68/>.
- Kay H. Brodersen, Cheng Soon Ong, Klaas E. Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *20th International Conference on Pattern Recognition*, pages 3121–3124. IEEE, 2010. doi: 10.1109/ICPR.2010.764. URL <https://doi.org/10.1109/ICPR.2010.764>.
- Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=ETKGubyOhcs>.
- Shenxu Chang, Junchi Yu, Weixing Wang, Yongqiang Chen, Jialin Yu, Philip Torr, and Jindong Gu. TRACEDET: Hallucination detection from the decoding trace of diffusion large language models. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=4puxTouUSV>.
- Chao Chen, Kai Liu, Ze Chen, Yi Gu, Yue Wu, Mingyuan Tao, Zhihang Fu, and Jieping Ye. INSIDE: LLMs’ internal states retain the power of hallucination detection. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Zj12nz1Qbz>.
- Yung-Sung Chuang, Linlu Qiu, Cheng-Yu Hsieh, Ranjay Krishna, Yoon Kim, and James R. Glass. Lookback lens: Detecting and mitigating contextual hallucinations in large language models using only attention maps. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 1419–1436, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.84. URL <https://aclanthology.org/2024.emnlp-main.84/>.
- Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning*. ACM, 2006. doi: 10.1145/1143844.1143874. URL <https://doi.org/10.1145/1143844.1143874>.
- Xuefeng Du, Chaowei Xiao, and Yixuan Li. Haloscope: Harnessing unlabeled LLM generations for hallucination detection. In *Advances in Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=nfK0ZXFFSn>.
- Sebastian Farquhar, Jannik Kossen, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024. doi: 10.1038/s41586-024-07421-0. URL <https://www.nature.com/articles/s41586-024-07421-0>.
- Shansan Gong, Mukai Li, Jiangtao Feng, Zhiyong Wu, and Lingpeng Kong. Diffuseq: Sequence to sequence text generation with diffusion models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=jQj-_rLVXsj.
- Zhengnan Guo and Fei Tan. Lost in diffusion: Uncovering hallucination patterns and failure modes in diffusion large language models. *arXiv preprint arXiv:2604.10556*, 2026. URL <https://arxiv.org/abs/2604.10556>.
- Raavi Gupta, Pranav Hari Panicker, Sumit Bhatia, and Ganesh Ramakrishnan. Consistency is the key: Detecting hallucinations in LLM generated text by checking inconsistencies about key facts. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 2053–2068, Mumbai, India, 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.findings-ijcnlp.129/>.
- Jiatong Han, Neil Band, Muhammed Razzak, Jannik Kossen, Tim G. J. Rudner, and Yarin Gal. Simple factuality probes detect hallucinations in long-form natural language generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025. URL <https://aclanthology.org/2025.findings-emnlp.880/>.
- Arshia Hemmat, Philip Torr, Yongqiang Chen, and Junchi Yu. TDGNet: Hallucination detection in diffusion language models via temporal dynamic graphs. *arXiv preprint arXiv:2602.08048*, 2026. URL <https://arxiv.org/abs/2602.08048>.

- Ganesh Jawahar, Benoît Sagot, and Djamé Seddah. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019. URL <https://aclanthology.org/P19-1356/>.
- Che Jiang, Biqing Qi, Xiangyu Hong, Dayuan Fu, Yang Cheng, Fandong Meng, Mo Yu, Bowen Zhou, and Jie Zhou. On large language models’ hallucination with regard to known facts. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1041–1053, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.60. URL <https://aclanthology.org/2024.naacl-long.60/>.
- Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147/>.
- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022. URL <https://arxiv.org/abs/2207.05221>.
- Jiyeon Kim, Sungik Choi, Yongrae Jo, Moontae Lee, and Minjoon Seo. Early decisions matter: Proximity bias and initial trajectory shaping in non-autoregressive diffusion language models. *arXiv preprint arXiv:2604.10567*, 2026. URL <https://arxiv.org/abs/2604.10567>.
- Jannik Kossen, Jiatong Han, Muhammed Razzak, Lisa Schut, Shreshth Malik, and Yarin Gal. Semantic entropy probes: Robust and cheap hallucination detection in LLMs. *arXiv preprint arXiv:2406.15927*, 2024. URL <https://arxiv.org/abs/2406.15927>.
- Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=VD-AYtP0dve>.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL <https://aclanthology.org/Q19-1026/>.
- Ping Li and Xiaoyun Li. Oporp: One permutation + one random projection. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2023. doi: 10.1145/3580305.3599457. URL <https://doi.org/10.1145/3580305.3599457>.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online, 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.acl-long.353. URL <https://aclanthology.org/2021.acl-long.353/>.
- Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori Hashimoto. Diffusion-LM improves controllable text generation. In *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=3s9IrEsjLyk>.
- Zhen Lin, Shubhendu Trivedi, and Jimeng Sun. Generating with confidence: Uncertainty quantification for black-box large language models. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=DWkJCSxKU5>.

- Zihang Liu, Jiawei Guo, Hao Zhang, Hongyang Chen, Jiajun Bu, and Haishuai Wang. Long-form hallucination detection with self-elicitation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 4082–4100, Vienna, Austria, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.211. URL <https://aclanthology.org/2025.findings-acl.211/>.
- Justin Lovelace, Varsha Kishore, Chao Wan, Eliot Seo Shekhtman, and Kilian Q. Weinberger. Latent diffusion for language generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 56998–57025, 2023. URL <https://openreview.net/forum?id=NKdtztladR>.
- Justin Lovelace, Varsha Kishore, Yiwei Chen, and Kilian Weinberger. Diffusion guided language modeling. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14936–14952, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.887. URL <https://aclanthology.org/2024.findings-acl.887/>.
- Andrey Malinin and Mark Gales. Uncertainty estimation in autoregressive structured prediction. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=jN5y-zb5Q7m>.
- Potsawee Manakul, Adian Liusie, and Mark Gales. SelfcheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.557. URL <https://aclanthology.org/2023.emnlp-main.557/>.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. URL <https://arxiv.org/abs/2502.09992>.
- Seongheon Park, Xuefeng Du, Min-Hsuan Yeh, Haobo Wang, and Yixuan Li. How to steer LLM latents for hallucination detection? In *International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2503.01917>.
- Yanyu Qian, Yue Tan, Yixin Liu, Wang Yu, and Shirui Pan. DynHD: Hallucination detection for diffusion large language models via denoising dynamics deviation learning. *arXiv preprint arXiv:2603.16459*, 2026. URL <https://arxiv.org/abs/2603.16459>.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. URL <https://arxiv.org/abs/2412.15115>.
- Vittorio Rossi, Giacomo Cirò, Davide Beltrame, Luca Gandolfi, Paul Röttger, and Dirk Hovy. Diffusion language models are natively length-aware. *arXiv preprint arXiv:2603.06123*, 2026. URL <https://arxiv.org/abs/2603.06123>.
- Albert Sawczyn, Jakub Binkowski, Denis Janiak, Bogdan Gabrys, and Tomasz Jan Kajdanowicz. FactSelfCheck: Fact-level black-box hallucination detection for LLMs. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 5603–5621, Rabat, Morocco, 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-eacl.296. URL <https://aclanthology.org/2026.findings-eacl.296/>.
- Ahmad Shapiro, Karan Taneja, and Ashok Goel. HALT: Hallucination assessment via log-probs as time series. *arXiv preprint arXiv:2602.02888*, 2026. URL <https://arxiv.org/abs/2602.02888>.
- Gaurang Sriramanan, Siddhant Bharti, Vinu Sankar Sadasivan, Shoumik Saha, Priyatham Kattakinda, and Soheil Feizi. LLM-Check: Investigating detection of hallucinations in large language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 34188–34216, 2024. URL <https://openreview.net/forum?id=LYx4w3CAgy>.
- Weihsang Su, Changyue Wang, Qingyao Ai, Yiran Hu, Zhijing Wu, Yujia Zhou, and Yiqun Liu. Unsupervised real-time hallucination detection based on the internal states of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14379–14391, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.854. URL <https://aclanthology.org/2024.findings-acl.854/>.

- Ian Tenney, Dipanjan Das, and Ellie Pavlick. Bert rediscovers the classical nlp pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019. URL <https://aclanthology.org/P19-1452/>.
- Dimitri von Rütte, Janis Fluri, Omead Pooladzandi, Bernhard Schölkopf, Thomas Hofmann, and Antonio Orvieto. Scaling behavior of discrete diffusion language models. *arXiv preprint arXiv:2512.10858*, 2025. URL <https://arxiv.org/abs/2512.10858>.
- Yujia Xie, Hanjun Dai, Minshuo Chen, Bo Dai, Tuo Zhao, Hongyuan Zha, Wei Wei, and Tomas Pfister. Differentiable top-k with optimal transport. In *Advances in Neural Information Processing Systems*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/ec24a54d62ce57ba93a531b460fa8d18-Abstract.html>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Brussels, Belgium, 2018. Association for Computational Linguistics. URL <https://aclanthology.org/D18-1259/>.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. URL <https://arxiv.org/abs/2508.15487>.
- Fujie Zhang, Peiqi Yu, Biao Yi, Baolei Zhang, Tong Li, and Zheli Liu. Prompt-guided internal states for hallucination detection of large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21806–21818, Vienna, Austria, 2025a. Association for Computational Linguistics. URL <https://aclanthology.org/2025.acl-long.1058/>.
- Jiaxin Zhang, Zhuohang Li, Kamalika Das, Bradley A. Malin, and Sricharan Kumar. SAC³: Reliable hallucination detection in black-box language models via semantic-aware cross-check consistency. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 15445–15458, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.1032. URL <https://aclanthology.org/2023.findings-emnlp.1032/>.
- Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. Siren’s song in the AI ocean: A survey on hallucination in large language models. *Computational Linguistics*, 51(4):1373–1418, 2025b. doi: 10.1162/coli.a.16. URL <https://aclanthology.org/2025.cl-4.9/>.
- Zhenliang Zhang, Xinyu Hu, Huixuan Zhang, Junzhe Zhang, and Xiaojun Wan. ICR probe: Tracking hidden state dynamics for reliable hallucination detection in LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17986–18002, Vienna, Austria, 2025c. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.880. URL <https://aclanthology.org/2025.acl-long.880/>.
- Yukun Zhao, Lingyong Yan, Weiwei Sun, Guoliang Xing, Chong Meng, Shuaiqiang Wang, Zhicong Cheng, Zhaochun Ren, and Dawei Yin. Knowing what LLMs DO NOT know: A simple yet effective self-detection method. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7051–7063, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.390. URL <https://aclanthology.org/2024.naacl-long.390/>.

A Additional Experimental Details

A.1 End-to-End Pipeline Overview

Our experiments follow a six-stage pipeline that instantiates HIVE from raw question-answer data to final hallucination detection metrics. At a high level, this pipeline refines the four-component

formulation in the main text—hidden evidence extraction, step-layer evidence selection, evidence-conditioned verification, and decision-logit scoring—into a reproducible sequence of preprocessing, training, export, and evaluation stages.

In the first stage, we run the target diffusion language model on each input question and record its denoising trajectory. Rather than storing the full hidden trajectory, we retain only a fixed number of recent denoising forwards and extract hidden states from a sparse set of candidate positions at each retained step, consisting of the last-token position and a capped set of changed-token positions. These hidden states are then compressed with OPORP-style random projection and written to memory-mapped feature files together with auxiliary metadata such as row ids, capture lengths, and forward ids.

In the second stage, we train a time-aware step-layer selector on top of the compressed trajectory features. For each candidate step-layer unit, we construct a two-stream representation that concatenates a last-token feature with a mean-pooled changed-token feature. The selector augments this two-stream evidence with learnable step and layer embeddings, scores all retained step-layer units jointly, and learns a top- K evidence policy under hallucination supervision. In practice, the selector is trained with a straight-through top- K mechanism and an entropy regularizer to encourage stable evidence selection.

In the third stage, we apply the trained selector to each split and export the selected evidence for downstream verification. For every example, we store the selected step-layer pairs, their top- K selector weights, and the alignment information needed to reconstruct the evidence units. When requested, we also export the selected compressed activations themselves as `act_top`, together with the derived two-stream evidence representation used by the verifier.

In the fourth stage, we pack the exported evidence into verifier training data. Each packed example contains the original question, the base model answer, the selected step-layer pairs, and a structured JSON target specifying the hallucination decision, hallucination type, evidence field, and rationale. This yields train/validation/test JSONL files for the verifier, while preserving pointers back to the exported evidence shards for efficient loading.

In the fifth stage, we train a verifier language model with ACT-prefix conditioning. The verifier uses Qwen2.5-7B-Instruct as the textual backbone. Each selected evidence unit is converted into a two-stream evidence vector, encoded into the verifier hidden space, and injected as a prefix embedding sequence before the textual question-answer context. The verifier is then trained to generate a schema-constrained structured output, allowing HIVE to perform verification rather than only binary scoring.

In the final stage, we evaluate the trained verifier on held-out examples. During inference, the verifier generates the structured JSON prediction, and we additionally extract the decision-emission logits associated with the binary decision field. These logits are converted into a continuous hallucination score, which is used to compute threshold-free metrics such as AUROC and AUPRC, while the structured prediction itself is used to evaluate decision accuracy, hallucination type accuracy, valid-JSON rate, and other field-level exact-match metrics.

The remaining subsections provide the implementation details for each part of this pipeline, including feature storage, selector training, exported evidence format, verifier supervision, evaluation protocol, and runtime hyperparameters.

A.2 Data Construction and Trajectory Feature Storage

We construct trajectory features before selector training by running the target D-LLM on each input question and recording a compact representation of its denoising process. In our implementation, the builder reads question-answer supervision from a tab-separated file in which each line contains a question and its gold answer list, split on the first tab only. For each example, the script formats the question into a chat-style prompt, runs the base diffusion language model to obtain a generated answer, and records the corresponding denoising trajectory together with the generated text and label metadata. The same builder supports both the native diffusion-generation interface and a local LLaDA generation branch, so that the trajectory-construction stage can be applied to both Dream and LLaDA-style models within a unified pipeline.

A.2.1 Retained trajectory states and candidate positions

Rather than storing the full hidden trajectory, we retain only a fixed number of recent denoising forwards and a sparse subset of token positions within each retained forward. In the current implementation, the default setting keeps the last $N = 8$ forwards and allows at most 16 changed-token positions per retained forward, so that the maximum number of captured positions is

$$\text{max_M} = 1 + \text{max_changed_positions} = 17,$$

where the extra slot corresponds to the last-token position. At each forward call, the builder compares the current input sequence with the previous one, identifies positions whose token ids have changed, removes the last-token position from that changed set if necessary, caps the remaining changed positions, and then forms the final capture set as the union of the capped changed positions and the current last-token position. This yields a sparse but trajectory-aware set of candidate locations that emphasizes actively revised positions while always preserving a compact final-state signal.

For every retained step-layer pair, the script collects the hidden vectors at the captured positions from all decoder layers. Internally, the hook collector stores these per-layer vectors in the form $[M, H]$, where M is the number of captured positions in that forward and H is the original hidden dimension. The collected states are then stacked into a tensor of shape $[L, M, H]$, where L is the number of retained layers. If a particular layer is missing a captured tensor for a forward, the builder pads that slot with zeros so that the packed trajectory representation remains rectangular and can be written efficiently to disk.

In addition to the trajectory features themselves, the builder writes a metadata record for each processed example to `meta.jsonl`. Each record stores the original dataset id, question text, gold answers, generated base answer, prompt and sequence lengths, generation settings, the observed number of forward calls, the retained forward ids, the kept changed positions, the kept capture positions, the OPORP configuration, and the corresponding `feature_row` index. When enabled, the same stage also attaches both a hard string-match label and an LLM-judge label with auxiliary fields such as hallucination type, confidence, suspect span, and parsed judge output. These metadata fields are later used to construct selector labels and verifier supervision targets.

A.2.2 OPORP compression and mmap storage format

To keep trajectory storage tractable, we compress the collected hidden vectors with OPORP-style random projection before writing them to disk. The builder creates one projection matrix per layer, using an orthogonalized Gaussian matrix scaled by $\sqrt{d/r}$, and either loads the matrices from a cached checkpoint or creates them on demand. In the current implementation, the default compressed evidence dimension is $r = 64$, with one OPORP checkpoint stored per (L, H, r, seed) configuration. The projected hidden states are stored in float16 format, which substantially reduces storage cost while preserving the relative structure needed by downstream selector training and evidence export.

The packed trajectory features are written as memory-mapped arrays under the run directory. The main tensor `features_act.f16.mmap` has shape $[S, N, L, \text{max_M}, r]$, where S is the number of processed examples in the current run window. The companion file `features_meta.json` records the tensor layout, file names, dtypes, and symbolic dimensions $(S, N, L, \text{max_M}, r)$, allowing downstream scripts to read the stored features without hard-coding shapes. This mmap-based design is used consistently by both selector training and evidence export, and makes it possible to resume interrupted preprocessing runs while preserving row-level alignment between feature tensors and metadata.

A.3 Step-Layer Selector Training

After constructing compressed trajectory features, we train a time-aware step-layer selector to identify the most informative hidden evidence units for downstream verification. The selector operates on the memory-mapped trajectory tensors produced in the preprocessing stage and learns, for each example, which retained step-layer pairs should be preserved under a fixed evidence budget. In our implementation, selector training is performed directly on the compressed evidence representation rather than on the original hidden states, which keeps training efficient while preserving the trajectory structure required for evidence selection.

A.3.1 Two-stream feature construction for selector training

The selector reads the memory-mapped evidence tensor `features_act.f16.mmap` of shape $[S, N, L, \text{max_M}, r]$ together with the corresponding capture lengths `features_cap_len.i16.mmap`. Here, S is the number of samples, N is the number of retained denoising forwards, L is the number of retained layers, and r is the compressed evidence dimension. For each retained step, the training dataset reconstructs a two-stream feature at every layer by taking (i) the compressed hidden vector at the last captured position and (ii) the mean-pooled compressed hidden vector over the changed-token positions. Concretely, if a retained forward contains m captured positions, the final slot is interpreted as the last-token feature and the first $m - 1$ slots are averaged to produce the changed-token feature. The resulting per-pair representation is therefore

$$u_{t,\ell} = [u_{t,\ell}^{\text{last}}, u_{t,\ell}^{\text{chg}}] \in \mathbb{R}^{2r},$$

and the full example is reshaped into a grid of size $N \times L$ or, equivalently, a flattened set of $T = NL$ candidate evidence units. In the current configuration, the default feature dimension is $2r = 128$ when $r = 64$.

Labels for selector training are obtained from the metadata file `meta.jsonl`. The training script streams the metadata, builds a mapping from original example id to the requested label field, and then aligns those labels with the feature rows using `features_row_ids.i32.mmap`. This design allows the same stored trajectory features to be reused under different supervision signals without rebuilding the entire trajectory cache. The default training script also supports stratified train/validation/test splitting at the feature-row level.

A.3.2 Time-aware joint gating and top-K selection

The selector itself is implemented as a joint gating model over all retained step-layer pairs. Given the two-stream feature tensor $X \in \mathbb{R}^{N \times L \times 2r}$ for one example, the model augments each pair feature with a learnable step embedding and a learnable layer embedding. More precisely, for each pair (t, ℓ) , the gate input is formed as

$$\tilde{u}_{t,\ell} = [u_{t,\ell}; a_t; b_\ell],$$

where a_t is the step embedding and b_ℓ is the layer embedding. A shared multilayer perceptron then produces one scalar score for each step-layer pair, yielding a score tensor of shape $[N, L]$, or $[T]$ after flattening. This architecture makes the selector explicitly time-aware while still sharing the same scoring function across all retained trajectory locations.

To obtain a fixed evidence budget, the model applies a softmax over all T pair scores and then uses a straight-through top- K mechanism. During training, the top- K indices are selected from the flattened score vector, but the hard mask is combined with the soft pair probabilities so that gradient information still flows through the full score distribution. The resulting normalized pair weights are denoted by w_{all} , and the selected top- K weights are denoted by w_{top} . These weights are then used to construct a single aggregated evidence representation

$$m = \sum_{(t,\ell)} w_{t,\ell} \tilde{u}_{t,\ell},$$

where $\tilde{u}_{t,\ell}$ is the step-layer-aware pair representation. This pooled representation is passed to a lightweight probe network that predicts the binary hallucination label for the example. In other words, selector training is not performed with a direct pair-ranking loss, but through end-to-end supervision that rewards evidence selections yielding a strong hallucination decision signal.

A.3.3 Training objective, regularization, and split protocol

The selector is trained with binary supervision using a sigmoid-based binary cross-entropy loss on the probe logits. When enabled, the training script additionally applies a positive-class reweighting term computed from the class counts in the current run directory, which is useful when hallucination labels are imbalanced. In addition to the classification loss, the implementation includes an entropy regularizer over the selected top- K weights,

$$\mathcal{L}_{\text{ent}} = - \sum_{j=1}^K w_j \log w_j,$$

and optimizes the combined objective

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda_{\text{ent}} \mathcal{L}_{\text{ent}}.$$

The entropy term is used to stabilize the evidence distribution during top- K training.

For each run, the script performs a stratified train/validation/test split whenever possible. The model is optimized with AdamW, and the training loop tracks validation metrics such as accuracy, F1, and AUROC. Early stopping is applied according to a configurable validation metric, and the best checkpoint is stored together with the feature shapes, split indices, and training history. In addition to the checkpoint itself, the training script writes a result JSON containing dataset statistics, hyperparameters and train/validation/test metrics.

A.4 Evidence Export and Verifier Dataset Construction

After training the step-layer selector, we export its selected evidence for downstream verifier training and evaluation. This stage serves as the bridge between selector-side trajectory modeling and verifier-side structured prediction: the selector identifies the top- K step-layer units for each example, and the exported evidence is then repackaged into a JSON-pair format that can be consumed by the verifier language model. In our pipeline, these two steps are implemented separately as evidence export and verifier dataset packing.

A.4.1 Exported evidence shards and stored fields

The export stage reads the trained selector checkpoint together with the memory-mapped trajectory features and applies the selector to a chosen split. For each example, it reconstructs the same two-stream representation used during selector training, computes the selector scores, and extracts the top- K step-layer pairs together with their corresponding normalized selector weights. The exported evidence is written in shard-wise `.npz` files, separately for train, validation, and test when split indices are available.

Each evidence shard stores the selected pair indices in several aligned forms, including the flattened pair index, the step ids, the layer ids, and the associated top- K selector weights. In addition, the export script records the forward id and capture-length information associated with each selected pair, so that the selected evidence remains aligned with the original trajectory representation. When requested, the exporter also saves the selected compressed activations themselves as `act_top`, together with a derived two-stream representation for each selected evidence unit. Optional fields such as full selector scores, normalized pair probabilities, and step- or layer-aggregated scores can also be included for analysis, but they are not required for the default verifier training path.

Overall, the exported shard format keeps the verifier interface compact: instead of exposing the full trajectory tensor, it retains only the selector-chosen evidence together with the minimal alignment information needed to reconstruct its step-layer identity and feature content.

A.4.2 Construction of verifier JSON-pair datasets

The packing stage converts the exported evidence into structured verifier examples. It first scans the exported evidence shards for a given split, then aligns each shard row with the corresponding metadata entry through the original example id stored in `meta.jsonl`. From that metadata record, the packer resolves the question text, the base model answer, and the supervision label used for verifier training.

For each example, the packer constructs a chat-style verifier input consisting of a system prompt and a user prompt. The user prompt contains the question, the base answer, and the list of selected (step, layer) pairs. The target is stored as a single JSON object containing a binary decision, a hallucination type, an evidence field that reproduces the selected pairs, and a short rationale. In the packed verifier format, the decision semantics are fixed as

$$0 = \text{wrong/hallucinated}, \quad 1 = \text{correct/non-hallucinated}.$$

This convention is used consistently during verifier training and evaluation.

The packed examples are written as `train.jsonl`, `val.jsonl`, and `test.jsonl`. Each row contains the chat messages, the target JSON string, and an evidence field that stores a relative pointer

to the source evidence shard together with the row index inside that shard. The evidence field also keeps the selected pair list and, when enabled, lightweight top- K metadata such as selector weights and capture lengths. This design allows the verifier dataset to remain compact while preserving a direct link back to the exported evidence shards for feature loading and debugging.

Taken together, the export and packing stages transform selector outputs into a reusable verifier-training interface: the selector contributes a sparse set of informative step-layer evidence units, and the packer turns those units into structured supervision examples for ACT-prefix-conditioned verification.

A.5 Verifier Training with ACT Prefixes

After packing the verifier dataset, we train a verifier language model that combines textual question-answer context with the exported hidden evidence. In our implementation, the verifier uses Qwen2.5-7B-Instruct as the backbone language model, while the selected top- K evidence units are injected through an ACT-prefix mechanism before the textual prompt. This design allows the verifier to condition its structured judgment not only on the final base answer, but also on the hidden evidence selected from the denoising trajectory.

A.5.1 ACT-prefix encoder

For each packed example, the verifier dataset loads the referenced evidence shard and reconstructs the top- K evidence representation. When the exported shard contains `act_top`, the loader rebuilds the same two-stream feature used by the selector, namely a concatenation of a last-token vector and a mean-pooled changed-token vector. If only a pooled evidence tensor is available, the loader falls back to a compatible two-stream representation. In either case, the final verifier-side evidence input has shape $[K, 2r]$ and is accompanied by the corresponding selector weights, step ids, and layer ids.

These top- K evidence vectors are then mapped into the verifier hidden space by an ACT-prefix encoder. The encoder first projects each evidence vector into the verifier model dimension and then fuses this projected representation with optional embeddings for layer identity, step identity, and selector weight. In the default configuration used in our codebase, all three side signals are enabled. The resulting evidence embeddings have shape $[K, d_{\text{model}}]$ and serve as the verifier-side ACT prefix.

A.5.2 Prefix-conditioned Qwen verifier

The verifier wrapper prepends the ACT prefix embeddings to the ordinary text token embeddings before passing the combined sequence to the base Qwen model. Concretely, the wrapper first tokenizes the verifier prompt built from the packed chat messages, then concatenates the ACT-prefix embeddings with the textual token embeddings, and finally extends the attention mask so that the model attends jointly to both the evidence prefix and the text tokens. During training, the prefix positions are excluded from the language-modeling loss by setting their labels to -100 , so supervision is applied only to the generated structured target sequence.

This wrapper is used in both training and inference. At training time, the prefix is injected only once at the beginning of the sequence; during autoregressive decoding, subsequent generation steps reuse the cached decoder state and do not repeatedly reinsert the prefix. In this way, the verifier behaves like a standard causal language model whose initial hidden context has been augmented with the selected trajectory evidence.

A.5.3 Training data format and supervision targets

Each verifier example consists of a system message, a user message containing the question, base answer, and selected step-layer pairs, and a target JSON string. The supervision target includes a binary decision field, a hallucination-type field, an evidence field reproducing the provided step-layer pairs, and a short rationale. The dataset tokenizes the prompt and target jointly, masks out the prompt portion in the labels, and trains the verifier to autoregressively generate only the structured JSON output.

In the current implementation, verifier training is performed with standard Hugging Face training utilities, optional LoRA adaptation, and mixed-precision support. The data collator pads the text sequence and batches the evidence tensors (x_{top} , w_{top} , `step_ids`, `layer_ids`) in parallel, so that

each training batch contains both the textual supervision signal and the aligned ACT-prefix evidence. This yields a verifier that is trained directly on evidence-conditioned structured prediction rather than on scalar classification alone.

A.6 Evaluation Protocol and Decision-Logit Scoring

We evaluate the trained verifier from two complementary perspectives: structured-output quality and scalar hallucination detection quality. At evaluation time, the verifier reads the packed JSONL examples together with their referenced evidence shards, reconstructs the top- K ACT-prefix evidence, and generates a structured JSON prediction under greedy decoding. The same evaluation pipeline is used across train, validation, and test splits, with the main reported results taken from held-out evaluation splits.

A.6.1 Structured-output parsing and field-level evaluation

For each example, the evaluation script first rebuilds the verifier prompt from the stored chat messages and reloads the corresponding top- K evidence from the exported shard. The verifier then generates a structured JSON prediction conditioned on both the textual prompt and the ACT-prefix evidence.

The generated text is parsed by extracting the first valid JSON object from the decoder output. Structured-output metrics are then computed by comparing the predicted JSON fields with the target JSON stored in the packed dataset. In particular, the evaluation script tracks valid-JSON rate, decision accuracy, hallucination-type accuracy, exact match on the provided evidence pairs, rationale exact match, and full exact match across all structured fields. This design allows us to evaluate HIVE not only as a detector, but also as a verifier that emits stable and interpretable structured outputs.

A.6.2 Decision-emission logit extraction

In addition to the structured JSON itself, we derive a continuous hallucination score from the verifier’s decision-emission logits. During greedy decoding, the evaluation script monitors the generated output until the "decision" field is reached. At the step where the model emits the binary decision token, the script collects the logits associated with tokenizations corresponding to the digits 0 and 1, aggregates them into a binary logit contrast, and converts the result into a probability p_1 for the event

$$\text{decision} = 1,$$

where 1 denotes a correct/non-hallucinated answer in the packed verifier format. The final hallucination score is then defined as

$$s_{\text{hall}} = 1 - p_1.$$

This score preserves finer-grained verifier confidence than the hard decision alone and serves as the scalar signal used for threshold-free hallucination detection evaluation.

A.6.3 Threshold selection and transferred evaluation

Given the continuous hallucination score, we compute threshold-free metrics such as AUROC and AUPRC directly from the full score distribution. For thresholded evaluation, the implementation supports several threshold-selection strategies, including best-F1 and Youden-style selection. In the main experiments, thresholds are selected on the validation split and then transferred unchanged to the test split, so that thresholded metrics reflect a realistic deployment-style setting rather than per-split re-optimization. Once a threshold is fixed, the evaluation script reports precision, recall, F1, accuracy, balanced accuracy, specificity, and the associated confusion matrix.

Taken together, this evaluation protocol yields two complementary views of verifier quality. The structured-output metrics assess whether the model generates valid and diagnostically meaningful verification fields, while the decision-logit score provides a continuous hallucination signal for standard detection metrics and downstream thresholded classification.

A.7 Training Hyperparameters and Runtime Configuration

We summarize the main training and runtime settings in Tables 4 and 5. Table 4 reports the default selector-training configuration used for time-aware two-stream step-layer gating, while Table 5

Parameter	Value
Optimizer	AdamW
Batch size	128
Epochs	30
Learning rate	1×10^{-3}
Weight decay	1×10^{-4}
Dropout	0.2
Train / val / test split	0.7 / 0.1 / 0.2
Top- K budget K	16
Step embedding dim	16
Layer embedding dim	16
Gate hidden dim	128
Probe hidden dim	512
Gate temperature τ	1.0
Entropy regularization λ_{ent}	1×10^{-3}
Positive-class reweighting	Optional
Early-stop patience	10
Early-stop metric	AUROC
Random seed	42

Table 4: **Selector training hyperparameters.** Default configuration of the time-aware two-stream step-layer selector used in the evidence-selection stage.

summarizes the verifier training and decoding settings used for ACT-prefix-conditioned structured verification. Unless otherwise specified, the experiments in the paper follow these configurations, with model-specific layer counts inherited from the underlying D-LLM.

B Prompt Templates and Output Schema

B.1 Verifier Prompt Format

HIVE uses a chat-style verifier prompt that combines ordinary textual context with an explicit schema instruction. Each verifier example is organized as a two-message conversation consisting of a system message and a user message. The system message defines the model as a verifier, explains that it should judge whether the base answer is wrong or hallucinated, and states that hidden evidence has already been injected through ACT-prefix embeddings. It also requires the model to output a single JSON object and forbids any extra natural-language text outside the schema.

The user message contains four main parts. First, it states the task, namely to judge whether the base answer is hallucinated with respect to the question. Second, it provides the original question. Third, it includes the base model answer to be verified. Fourth, it presents the selected hidden evidence in symbolic form as a list of top- K (step, layer) pairs. In other words, although the verifier does not directly read the full denoising trajectory in text form, the prompt still exposes the identities of the selected evidence units that correspond to the injected ACT prefixes.

The prompt additionally includes an explicit output-schema specification. This schema tells the verifier that it must produce a JSON object containing a binary decision field, a hallucination-type field, an evidence field that reproduces the provided step-layer pairs, and a short rationale. In the packed verifier format, the decision semantics are fixed as

$$0 = \text{wrong/hallucinated}, \quad 1 = \text{correct/non-hallucinated}.$$

This convention is stated directly in the prompt so that the verifier is trained and evaluated under a consistent decision interface.

Overall, the verifier prompt is designed to do two things simultaneously: it provides the usual textual verification context through the question and base answer, and it anchors the hidden-evidence interface by exposing the selected step-layer pairs and the required structured output format. This makes the verifier behave as a schema-constrained evidence-conditioned judge rather than as a free-form text generator.

Parameter	Value
Verifier backbone	Qwen2.5-7B-Instruct
Retained steps N	8
Compressed dimension r	64
Top- K evidence budget	16
Maximum input length	1024
Padding multiple	8
Train / val / test split	0.7 / 0.1 / 0.2
Batch size	1
Gradient accumulation	8
Epochs	4
Learning rate	2×10^{-5}
Weight decay	0.0
Warmup ratio	0.03
Logging steps	10
Save steps	500
Eval steps	500
Training dtype	auto (bf16/fp16 optional)
Device map (training)	none
NPZ cache size	8
LoRA	Optional
LoRA rank / alpha / dropout	8 / 16 / 0.05
ACT-prefix ablation flag	optional (disable_prefix_act)
Evaluation split default	test
Maximum decoding length	256
Inference dtype default	bf16
Threshold strategy default	best-F1
Fixed / transferred threshold	supported

Table 5: **Verifier training and decoding hyperparameters.** Default configuration of ACT-prefix verifier training and structured evaluation.

B.2 Output JSON Schema

The verifier is trained to generate a single JSON object with a fixed schema. This schema is explicitly specified in the verifier prompt and is also used as the supervision target during training. In our implementation, each target JSON contains four top-level components: a binary decision field, a hallucination-type field, an evidence field, and a short rationale. The evidence field itself contains a single subfield, `pairs`, which reproduces the provided top- K (step, layer) indices exactly.

The schema can be summarized as follows:

```
{
  "decision": 0 or 1,
  "hallucination_type": one of
    ["none", "factual_error", "fabrication", "unsupported",
     "incomplete", "irrelevant", "reasoning_error", "other"],
  "evidence": {
    "pairs": the provided (step, layer) pairs
  },
  "rationale": a single short sentence
}
```

Among these fields, `decision` is the primary binary verification output. In the packed verifier format, the decision semantics are fixed as

$$0 = \text{wrong/hallucinated}, \quad 1 = \text{correct/non-hallucinated}.$$

The `hallucination_type` field refines this binary judgment into a more specific error category. The `evidence.pairs` field reports the selected top- K step-layer pairs into the output, and `rationale` provides a short natural-language justification.

This schema is used consistently throughout the pipeline. During dataset packing, the target JSON is serialized into the `target` field of each verifier example. During training, the verifier learns to autoregressively generate this structured object. During evaluation, the same schema is used for field-level comparison, including decision accuracy, hallucination-type accuracy, evidence-pair exact match, rationale exact match, and full exact match across all structured fields.

Overall, the output schema serves two purposes at once: it provides a standard binary hallucination decision for detection, and it exposes a richer verification interface that makes HIVE’s predictions more interpretable and easier to analyze than a single scalar score alone.

B.3 Hallucination Type Taxonomy

In addition to the binary decision field, HIVE predicts a hallucination-type label that refines the verifier output into a more specific error category. In the current schema, the allowed type set is `{none, factual_error, fabrication, unsupported, incomplete, irrelevant, reasoning_error, other}`. These type labels are specified directly in the verifier prompt and are also used as the supervision targets during training.

The label `none` is reserved for non-hallucinated answers, namely cases in which the verifier judges the base answer to be correct. All remaining labels correspond to different kinds of hallucinated or otherwise unreliable answers. Among them, `factual_error` refers to answers that contain a concrete incorrect fact, entity, number, or relation; `fabrication` refers to answers that invent unsupported content more substantially; `unsupported` refers to answers that are not adequately supported with respect to the question; `incomplete` refers to answers that are partially correct but omit an important required element; `irrelevant` refers to answers that do not address the question; and `reasoning_error` refers to answers whose final conclusion is wrong because of an incorrect reasoning process. The residual category `other` is used only when the error does not fit cleanly into the preceding types.

This taxonomy is intended to provide a lightweight but useful decomposition of verifier behavior. It is not meant to be an exhaustive theory of hallucination, but rather a practical structured label space that allows us to evaluate whether the verifier captures more than a binary correct-versus-hallucinated distinction. In particular, hallucination-type prediction is reported separately from decision accuracy in the main paper, making it possible to assess whether HIVE can produce diagnostically meaningful structured judgments in addition to strong scalar detection performance.

In the packed verifier format, the hallucination type must remain consistent with the binary decision. In particular, when `decision = 1`, the corresponding type should be `none`; otherwise the type should belong to one of the hallucination categories listed above. This consistency is enforced at the schema level during dataset construction and is checked again during structured-output evaluation.

B.4 Verifier Dataset Example

Each verifier example is stored as one JSONL record containing the chat-style prompt, the structured target JSON, and the evidence pointer needed to reload the exported top- K evidence shard. For readability, we show below a simplified example that keeps only the fields most relevant to verifier supervision and omits auxiliary bookkeeping fields unrelated to the prompt or target format.

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a verifier model. Given a question and a base model answer, decide whether the answer is wrong/hallucinated. ... You MUST output a single JSON object ..."
    },
    {
      "role": "user",
      "content": "[Task] Judge whether the Base Answer is wrong/hallucinated with respect to the Question."
    }
  ]
}
```

```

[Question]
Who was the beautiful sister of the
twins Castor and Pollux and mother to Hermione?

[Base Answer]
The beautiful sister of the twins Castor
and Pollux, and mother to Hermione,
was Leda (or Leta) in Greek mythology.

[Evidence]
You received K=16 ACT token-groups
as hidden evidence. The (step, layer)
pairs of these ACT token-groups are:
[[2, 27], [3, 27], ..., [0, 24]]

[Output JSON Schema]
{
  "decision": 0 or 1,
  "hallucination_type": one of ["none","factual_error","fabrication",
  "unsupported", "incomplete","irrelevant",
  "reasoning_error","other"],
  "evidence": {
    "pairs": the provided (step, layer) pairs
  },
  "rationale": a single short sentence
}
Return only the JSON."}],
"target": {
  "decision": 0,
  "hallucination_type": "factual_error",
  "evidence": {
    "pairs": [[2, 27], [3, 27], ..., [0, 24]]
  },
  "rationale": "Based on the provided hidden
evidence, the base answer is likely
hallucinated; the suspected erroneous
span is: 'Leda (or Leta)'."
}
}

```

This example illustrates the basic verifier supervision format. The textual prompt provides the question, the base answer, and the symbolic identities of the selected evidence units, while the target specifies the structured verification output that the model must generate. In the packed dataset, each record also includes an evidence field containing a relative pointer to the source evidence shard and the row index within that shard, so that the corresponding ACT-prefix features can be loaded during training and evaluation.

Two aspects of this format are particularly important. First, the target explicitly reproduces the selected step-layer pairs in the evidence.pairs field, which ties the generated verification output back to the hidden-evidence interface. Second, the rationale remains short and schema-constrained rather than open-ended, so the verifier is trained to emit compact verification summaries instead of unrestricted explanations. This keeps the structured output stable and makes field-level evaluation straightforward.

B.5 Structured Output Parsing and Evaluation Rules

At inference time, the verifier generates a text sequence that is expected to contain exactly one structured JSON object. The evaluation pipeline first decodes the generated tokens into text and then extracts the first valid JSON object from the output. If no valid JSON object can be recovered, the

Method	Dream-7B-Instruct			LLaDA-8B-Instruct		
	HotpotQA	NQOpenLike	TriviaQA	HotpotQA	NQOpenLike	TriviaQA
Perplexity	65.28 / 85.36	60.13 / 87.00	65.75 / 75.54	53.34 / 76.05	59.16 / 87.16	50.42 / 64.29
LN-Entropy	67.31 / 88.04	65.87 / 90.55	67.05 / 77.36	65.07 / 85.47	72.20 / 92.23	68.35 / 79.19
Semantic Entropy	81.84 / 93.10	72.17 / 91.47	81.92 / 86.92	78.00 / 90.04	81.24 / 94.57	80.98 / 86.79
Lexical Similarity	74.29 / 90.42	66.21 / 88.74	70.57 / 79.64	64.40 / 83.16	73.06 / 92.40	68.31 / 78.51
EigenScore	74.46 / 90.33	68.58 / 89.76	72.56 / 80.34	75.07 / 89.26	76.62 / 93.47	74.15 / 83.12
CCS	89.60 / 96.65	81.31 / 92.98	88.75 / 91.22	87.79 / 92.98	81.46 / 95.09	84.33 / 89.95
TSV	78.47 / 92.32	70.28 / 92.28	79.37 / 87.21	76.16 / 89.53	71.86 / 92.94	73.05 / 83.02
TraceDet	87.40 / 95.64	79.47 / 94.86	83.44 / 88.60	86.05 / 94.28	77.68 / 94.16	80.07 / 86.89
HIVE	91.76 / 97.37	85.74 / 96.60	92.36 / 95.37	90.94 / 96.48	85.71 / 96.95	90.66 / 93.98

Table 6: **Full overall hallucination detection results.** Comparison against all baselines across two D-LLMs and three QA benchmarks. Each cell reports AUROC / AUPRC (% , higher is better).

prediction is treated as invalid for structured evaluation. This gives rise to the valid-JSON metric reported in the main paper, which measures whether the verifier successfully follows the required output format.

Once a valid JSON object is obtained, the prediction is compared against the target JSON stored in the packed dataset. The structured evaluation focuses on four fields: `decision`, `hallucination_type`, `evidence.pairs`, and `rationale`. The `decision` field is evaluated by exact agreement with the target binary label, the `hallucination_type` field is evaluated by exact agreement with the target type label, and the `rationale` field is evaluated by exact string match. For the `evidence.pairs` field, the prediction must reproduce the provided top- K (step, layer) pairs exactly and in order.

Based on these field-level comparisons, we report structured-output metrics including valid-JSON rate, decision accuracy, hallucination-type accuracy, evidence-pairs exact match, rationale exact match, and full exact match. The full-exact metric is the strictest: it requires all evaluated structured fields to match the target simultaneously. This makes it possible to distinguish between partial success, such as producing a correct decision but an incorrect rationale, and fully correct structured verification output.

An important aspect of this evaluation protocol is that the structured prediction is assessed independently of the continuous hallucination score used for AUROC and AUPRC. In other words, structured-output evaluation answers the question of whether the verifier produces a valid and diagnostically meaningful JSON prediction, while scalar detection evaluation measures how well the model ranks hallucinated and non-hallucinated answers. These two views are complementary, and together they capture both the detection quality and the verification quality of HIVE.

C Complete Result Tables

This appendix section reports the complete result tables corresponding to the compact summaries shown in the main paper. Specifically, we provide the full overall hallucination detection results, the full thresholded classification results, the full structured verification results, and the complete metric breakdowns for all ablation groups. These tables are included here to keep the main paper focused on the most important comparisons while preserving complete numerical coverage for reference.

D Asset Sources and Licenses

Table 10 summarizes the external models and datasets used in this work, together with their original sources and applicable licenses or usage terms. We cite the original creators of all such assets in the main paper and use them only for research and evaluation purposes within the scope of their official release terms.

Method	Dream-7B-Instruct				LLaDA-8B-Instruct			
	HotpotQA		TriviaQA		HotpotQA		TriviaQA	
Perplexity	87.19	/ 50.47 / 1.42	76.04	/ 49.99 / 0.00	85.99	/ 50.02 / 0.07	78.23	/ 50.44 / 1.42
LN-Entropy	87.27	/ 50.01 / 0.02	76.04	/ 49.99 / 0.00	85.96	/ 50.42 / 1.21	78.20	/ 52.66 / 8.43
Semantic Entropy	88.11	/ 64.63 / 35.87	80.30	/ 70.49 / 54.87	85.99	/ 50.00 / 0.00	82.23	/ 74.03 / 64.43
Lexical Similarity	87.37	/ 50.25 / 0.55	76.35	/ 51.80 / 4.97	86.49	/ 58.07 / 20.13	79.76	/ 60.53 / 28.28
EigenScore	87.61	/ 54.07 / 9.80	77.59	/ 59.19 / 25.22	86.42	/ 57.20 / 17.98	79.59	/ 59.04 / 24.05
CCS	90.53	/ 73.72 / 53.54	83.94	/ 79.18 / 74.36	90.30	/ 74.13 / 53.45	83.58	/ 73.51 / 58.78
TSV	87.83	/ 60.36 / 25.37	79.02	/ 64.99 / 39.98	86.74	/ 57.30 / 17.41	79.16	/ 55.79 / 15.16
TraceDet	89.78	/ 76.48 / 62.68	81.92	/ 71.41 / 52.88	86.23	/ 51.04 / 2.10	78.25	/ 50.01 / 0.02
HIVE	91.47	/ 77.77 / 61.96	87.76	/ 83.63 / 78.53	91.20	/ 79.39 / 65.53	88.07	/ 83.27 / 78.41

Table 7: **Full thresholded classification results.** Comparison against all baselines on HotpotQA and TriviaQA. Each cell reports F1 / Balanced Accuracy / Specificity (% , higher is better).

Metric	Dream-7B-Instruct			LLaDA-8B-Instruct		
	HotpotQA	NQOpenLike	TriviaQA	HotpotQA	NQOpenLike	TriviaQA
Valid JSON Rate	100.00	99.83	100.00	100.00	99.43	100.00
Decision Accuracy	86.46	83.88	85.29	86.49	83.18	84.55
Hallucination Type Accuracy	66.24	61.52	69.76	70.82	67.39	72.43
Full Exact Match	58.87	47.34	65.86	53.60	31.11	60.49

Table 8: **Full structured verification results of HIVE.** Structured verification quality across all six model–benchmark settings (% , higher is better). Valid JSON Rate measures whether the verifier output is syntactically well-formed. Decision Accuracy measures correctness of the hallucination decision field. Hallucination Type Accuracy measures correctness of the predicted hallucination type. Full Exact Match requires the full structured output to match the reference exactly.

E Compute Resources

All experiments were conducted on NVIDIA H100 80GB GPUs, with up to 10 GPUs used concurrently across runs. Table 11 reports the approximate wall-clock time of the main stages in the HIVE pipeline. These values are intended as reproducibility-oriented estimates and may vary across model–benchmark settings.

For a single model–benchmark setting, the full HIVE pipeline therefore required approximately 70–190 hours of end-to-end wall-clock time. Under a single-GPU accounting view, this corresponds to roughly 70–190 H100-GPU-hours per setting. Since the main paper reports six model–benchmark settings, the main experiments required at least approximately 420–1140 GPU-hours in total, excluding pilot runs, failed runs, and routine debugging overhead.

Variant	AUROC	AUPRC	F1	BAcc	Spec	DecAcc
<i>Ablation 1: Hidden-Evidence Prefix Conditioning</i>						
HIVE (full)	92.36	95.37	87.76	83.63	78.53	85.29
Plain verifier, no evidence prefix (retrained)	88.54	91.75	84.25	78.28	74.62	81.25
Prefix-trained, no prefix at inference	86.64	90.03	82.26	75.51	70.11	77.07
Shuffled evidence prefix	88.57	90.75	84.28	78.17	74.30	81.18
<i>Ablation 2: Learned Step-Layer Evidence Selection</i>						
HIVE (full learned selector)	92.36	95.37	87.76	83.63	78.53	85.29
Top-final-step-only	89.17	93.39	85.82	79.27	73.25	81.94
Uniform-over-steps	88.34	93.22	83.92	81.00	73.44	80.54
Random K pairs	85.85	90.44	83.05	77.73	66.59	78.81
<i>Ablation 3: Two-Stream Evidence Representation</i>						
Two-stream full [u^{last} ; u^{chg}]	92.36	95.37	87.76	83.63	78.53	85.29
Lasttok only	90.48	93.36	86.63	81.00	76.45	83.64
Changed only	89.93	92.85	85.50	80.03	76.42	82.37
<i>Ablation 4: Step-Layer Embeddings</i>						
Full (step + layer embeddings)	92.36	95.37	87.76	83.63	78.53	85.29
w/o step embedding	90.93	93.69	85.74	82.47	76.00	82.95
w/o layer embedding	91.43	94.88	86.62	82.79	76.38	83.49
<i>Ablation 5: Decision-Logit Hallucination Scoring</i>						
Decision-logit score	92.36	95.37	87.76	83.63	78.53	85.29
Binary decision only	–	–	86.53	82.69	77.95	84.94

Table 9: **Full ablation results.** Complete metric breakdown for all ablation groups on Dream-7B-Instruct + TriviaQA. Metrics include AUROC, AUPRC, F1, Balanced Accuracy (BAcc), Specificity (Spec), and Decision Accuracy (DecAcc), all reported in % (higher is better).

Asset	Type	Source / Citation	License / Terms	Usage in this paper
Dream-v0-Instruct-7B	Model	Ye et al. [2025]	Apache-2.0	Target D-LLM backbone used for trajectory extraction and evaluation.
LLaDA-8B-Instruct	Model	Nie et al. [2025]	MIT	Target D-LLM backbone used for trajectory extraction and evaluation.
Qwen2.5-7B-Instruct	Model	Qwen Team [2025]	Apache-2.0	Verifier backbone used for evidence-conditioned structured verification.
TriviaQA	Dataset	Joshi et al. [2017]	Apache-2.0	Evaluation benchmark for open-domain question answering.
HotpotQA	Dataset	Yang et al. [2018]	CC BY-SA 4.0	Evaluation benchmark for multi-hop question answering.
Natural Questions	Dataset	Kwiatkowski et al. [2019]	Apache-2.0	Source benchmark family underlying the NQ-style evaluation setting.
NQOpenLike	Derived evaluation setting	Derived from the Natural Questions benchmark family [Kwiatkowski et al., 2019]	Subject to the license and terms of the underlying Natural Questions source data	Internal NQ-style open-ended QA evaluation setting used in this work.

Table 10: **External assets used in this paper.** We cite the original sources of all external models and datasets used in our experiments and summarize their licenses or usage terms where applicable. For the derived NQOpenLike setting, the relevant terms follow those of the underlying source data.

Stage	Hardware	Approx. wall-clock time
Trajectory feature construction	1×H100 80GB	40–70 h
Selector training	1×H100 80GB	5–20 min
Evidence export / dataset packing	1×H100 80GB	~1 min
Verifier training	1×H100 80GB	20–90 h
Evaluation	1×H100 80GB	10–30 h

Table 11: **Approximate compute usage by pipeline stage.** Reported times are per model–benchmark setting.